

B.E. Even Semester Midterm Examination, 2024-25

Subject: Data Structures / Design and Analysis of Algorithms

Q1. Find the MST from the following graph using Kruskal algorithm

(Assuming graph nodes based on the diagram: Left triangle A,B,C; Right region D,E,F joined at C. Edges: AB=1, AC=2, BC=3, CD=4, CE=5, CF=6, DE=7, EF=8)

Answer:

Kruskal's Algorithm finds the Minimum Spanning Tree (MST) by sorting all edges in non-decreasing order of their weights and adding them one by one to the MST, provided they do not form a cycle.

The graph has 6 vertices . A valid MST requires exactly edges.

Step 1: Sort edges by weight:

- AB (weight 1)
- AC (weight 2)
- BC (weight 3)
- CD (weight 4)
- CE (weight 5)
- CF (weight 6)
- DE (weight 7)
- EF (weight 8)

Step 2: Add edges avoiding cycles:

- Add **AB (1)**. MST Edges:
- Add **AC (2)**. MST Edges:
- Reject BC (3) as it creates the cycle A-B-C.
- Add **CD (4)**. MST Edges:
- Add **CE (5)**. MST Edges:
- Add **CF (6)**. MST Edges:

We have 5 edges. The algorithm terminates.

Minimum Spanning Tree edges: AB, AC, CD, CE, CF

Total MST Weight:

Q2. Breadth first traversal of a graph uses representation of graph in which data structure? Find the BFS sequence for the following graph.

(Graph nodes: M, N, O, P, Q, R. Edges: M-N, M-R, M-Q, N-O, N-Q, O-Q, O-P, P-Q)

Answer:

- **Data Structure:** Breadth-First Search (BFS) utilizes a **Queue** (First-In-First-Out) data structure to keep track of the vertices that need to be explored next.
- **BFS Sequence:** (Assuming we start traversal at alphabetical root, Node M, and visit neighbors in alphabetical order).
 - Start at **M**. Mark visited. Enqueue M. Queue: [M].
 - Dequeue M. Output: **M**. Enqueue unvisited neighbors of M: N, Q, R. Queue: [N, Q, R].
 - Dequeue N. Output: **N**. Enqueue unvisited neighbors of N: O. Queue: [Q, R, O].
 - Dequeue Q. Output: **Q**. Enqueue unvisited neighbors of Q: P. (O is already in queue). Queue: [R, O, P].
 - Dequeue R. Output: **R**. No unvisited neighbors. Queue: [O, P].
 - Dequeue O. Output: **O**. No unvisited neighbors. Queue: [P].
 - Dequeue P. Output: **P**. Queue is empty.

BFS Sequence: M, N, Q, R, O, P (Note: Sequences may vary slightly depending on the neighbor visitation order, e.g., M, N, R, Q... but level by level must be maintained).

Q3. Why are all Master's theorems compared with the function ?

Answer:

The Master Theorem provides a way to solve recurrence relations of the form .

When expanding this recurrence into a recursion tree:

- The tree has a depth of .
- At the bottom level (the leaves of the tree), there are leaves.
- By logarithmic identities, .

The term $T(n)$ represents the **total amount of work done at the leaf level** of the recursion tree (base cases).

The function $f(n)$ represents the work done at the root node (splitting and combining).

Master's theorem compares $T(n)$ with $f(n)$ to determine which part of the tree dominates the computational cost:

- If leaf work dominates ($T(n)$ polynomially), the complexity is $\Theta(T(n))$.
- If the work is evenly balanced ($T(n) = \Theta(n^k)$), the complexity is multiplied by a logarithmic factor: $\Theta(n^k \log n)$.
- If root/combining work dominates ($f(n)$ polynomially), the complexity is $\Theta(f(n))$.